If the developers work collaboratively with testers writing end-to-end tests inside their project, then consumer contracts will be generated within the frontend project and can be used as mocks in the Cypress tests directly from Pact's output folder. The Cypress example in listing 9.1 and 9.2 isn't the most optimal way to use contract mocks; however, it's a nice solution if the end-to-end tests currently use static mocks.

Another feature we're going to touch on in this chapter is provider verification badges. As we mentioned earlier in section 9.3.3 regarding user management, some users will solely interact with Pact Broker through the CLI within their CI pipelines. Within the CI pipeline, a great way to visualize the status of the tests is through a status badge. PactFlow provides status badges for each contract between services, as shown in 9.14. Display the provider verification status badge within a README so those running the CI pipeline can see the information they need.

Figure 9.14 Provider verification status badge to display Pact Broker contract status within the CI pipeline

9.4 Self-hosted Pact Broker

There are a few different options for spinning up the open source version of Pact Broker, depending on your existing infrastructure and potential security requirements or maintenance strategy. In this section, we'll set up Pact Broker using the following:

- Docker
- Kubernetes

It's possible to set up Pact Broker from scratch with nginx, PostgreSQL, and Ruby, where instructions are available on the Pact foundations GitHub (https://mng.bz/V240).